arXiv:submit/8077306 [cs.RO] 1 Oct 2026

REAP: Learning General Rearrangement Heuristics from Motion-Planner Feedback

Daniel Swoboda **Hector Geffner**
Chair of Machine Learning and Reasoning
RWTH Aachen University, Germany
{daniel.swoboda, hector.geffner}@ml.rwth-aachen.de

Abstract: Robotic rearrangement tasks require planning sequences of actions that fulfill high-level specifications while also being geometrically feasible. Symbolic abstractions make the combinatorial structure of rearrangement tasks tractable, but they usually do not fully capture the geometric constraints that determine which actions a robot can actually realize. This is due to the complexity of expressing the underlying geometric feasibility constraints in the symbolic model. We propose REAP (Rearrangement with Elicited Action Preconditions), a framework in which a relational graph neural network heuristic is learned over a simple symbolic abstraction. A motion planner serves as a supervisor that elicits the *hidden geometric preconditions* of the problem during training. Deployed with greedy best-first search, the learned heuristic generalizes out-of-distribution along object count, plan length, and object arrangement. Compared to classical heuristic search, TAMP, and learning-based baselines, REAP solves more problems, and requires roughly an order of magnitude fewer motion-planner queries than comparable learning-based methods. REAP anticipates geometric feasibility, suggesting geometrically feasible actions in 98% of instances. We show that the same architecture, training recipe, and search procedure adapt to various robotic rearrangement domains, with cross-domain generalization and transfer capabilities. Code, trained weights, datasets, baseline reimplementations, and the simulation environments are available at https://elicited-preconditions.github.io.

Keywords: Task and Motion Planning, Generalized Planning, RL

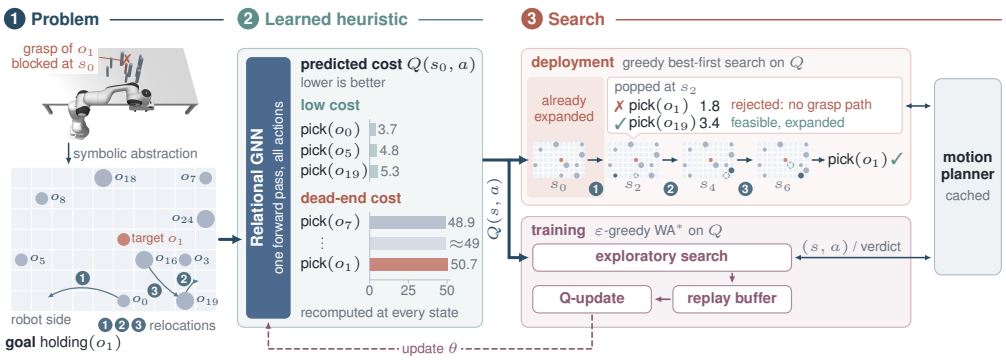


Figure 1: REAP learns which symbolically applicable actions a robot can actually execute. **(1)** Grasping the target o_1 is symbolically applicable but the motion planner cannot execute it. The symbolic abstraction of the initial state and goal is passed to a relational GNN. **(2)** Given the state, goal and symbolically applicable actions, a relational GNN predicts a cost $Q(s, a)$ for every action in one pass. REAP predicts accurately that relocation tasks have low cost, whereas infeasible picking tasks have high costs. **(3)** At deployment, greedy best-first search on Q is used to find a plan to get to the goal. The motion planner verifies each expanded action and rejects infeasible ones. During training, an ε-greedy WA* search stores the planner's verdicts in a replay buffer. The high cost of infeasible actions and the cost-to-go are used to update the GNN weights.

1 Introduction

Robots performing tasks in cluttered environments like clearing a table, retrieving an occluded target, or constructing an object from parts must select *what* to do symbolically and *how* to realize each step as a collision-free motion. These tasks are examples of combined task and motion planning (TAMP) [1, 2], where a symbolic plan reaching the given goal is sought and every action of that plan must be geometrically feasible. The simplest approach incrementally searches for plans while checking the feasibility of each candidate with a motion planner. This, however, is too inefficient as the geometric constraints play only a passive role in the search [3, 4]. Other approaches make the geometric constraints explicit in the planning formulation, but modeling such constraints and solving the resulting joint problem is similarly hard [5, 6]. A third line of work aims to amend the heuristics used to guide the search to take into account both the goals and the geometric constraints. The resulting search is more informed, and the challenge is the formulation and computation of such hybrid heuristics [7, 8, 9, 10].

In this work, we propose a novel approach for addressing robot rearrangement tasks based on heuristic estimators leveraging recent progress on generalized planning, where policies are learned for solving large collections of classical planning problems [11, 12, 13, 14]. Some of these methods learn these policies in terms of parametric value functions $Q_\theta(s, a)$ represented in relational graph neural networks (R-GNN) and trained without supervision using reinforcement learning [15, 16]. As in goal-conditioned RL [17, 18, 19], the atoms in s represent both the current state and the goal of the instance.

Generalized symbolic planning policies are particularly relevant in this setting in two ways. On the one hand, a single policy generalizes across instances varying in object count, initial state, and goal by acting greedily on the learned Q-value function. On the other, the policy can be learned *model-free*: it does not need to know the action preconditions and effects a priori. The training loop only requires a simulator that, for each state s, exposes the applicable action set $A(s)$, the successor $s' = f(a, s)$, an action cost $c(a, s)$, and a goal check.[1] What distinguishes TAMP from classical planning, given a suitable symbolic abstraction, is that not every symbolically applicable action is *geometrically executable*. Alongside $A(s)$ there is a hidden function $X(s) \subseteq A(s)$ encoding the set of *feasible* actions, decidable by the motion planner provided s encodes a suitable abstraction of the geometry. We illustrate this in Figure 1 on one of our rearrangement tasks where $\sim$80% of the symbolically applicable actions are not executable. Making the symbolic policy aware of this hidden function is a minor modification to the RL algorithm: selected actions $a \in A(s)$ found not to be executable in s, i.e., $a \in A(s) \setminus X(s)$, are penalized in the state s. The resulting Q-values will then learn to predict costs that combine the number of steps to the goal and their feasibility.

In order to have a complete TAMP planner, REAP (Rearrangement with Elicited Action Preconditions) *uses* the learned Q-values not to push greedily for the goal, but to guide a greedy best-first search [20, 21], over a coarse abstraction of the geometry, that uses grid-based adjacency relations and occupancy. The learned heuristic solves more problems than baselines, generalizes out-of-distribution along object count, plan length, and arrangement, and requires an order of magnitude fewer motion-planner queries at run time than classical heuristic search and learning-based baselines. Removing the motion-planner feedback collapses the framework to symbolic search, like our BrFS and h_{FF} baselines, but guided by the learned $Q_\theta(s, a)$ values. We demonstrate this approach across five rearrangement environments with distinct hidden-precondition structures, with no architectural or training-loop changes between them.

We contribute REAP, a TAMP framework that (i) trains a generalized TAMP planning heuristic with motion-planner verdicts as the RL signal, (ii) requires an order of magnitude fewer motion-planner queries at deployment, (iii) scales without retraining to OOD instances, and (iv) applies the same approach across five robotic rearrangement environments.

[1] Even a uniform-random policy needs these signals: knowing which actions are applicable, advancing the state, and detecting the goal is the minimum any policy needs to act.

2 Related Work

Integrating planning, learning, and execution is a long-standing problem in robotics, where a symbolic plan is only useful insofar as it grounds out in physically executable motion. Prior work approaches this from several directions, which we group below. Related work on **generalized planning** was discussed in the introduction.

TAMP with Learned Components. Classical task-and-motion planning couples symbolic search with parameter sampling and feasibility checks in an integrated search loop [3, 1, 2]. Various works attempt to replace parts of this stack with learned components. The closest to ours is Khodeir et al. [9], a GNN heuristic guiding search in a stream-based TAMP planner [1]. They use offline learning from past execution traces to prioritize the sampling of numerical parameters for the planners (modeled as streams). Another predecessor of our work is Wells et al. [7]. They learn an SVM-based binary feasibility classifier from offline trajectories with motion planner feedback, and use it to prune candidate task plans before invoking the motion planner. Other supervised approaches predict action feasibility from visual observations [8], or learn to sample trajectories with diffusion conditioned on the symbolic plan [22]. Yang et al. [10] move from single-action feasibility classifiers to predicting feasibility for an entire candidate plan skeleton. Curtis et al. [23] use LLMs to generate continuous CSPs from the task description to constrain the search space. Closest to REAP is Kim and Shimanuki [24], who also learn a relational value function with a GNN to guide a TAMP planner. Their approach (SAHS) differs in where geometric feasibility enters. SAHS encodes it in the state representation, through crafted predicates defined over swept volumes that require inverse kinematics and motion planning to evaluate. We learn a heuristic to guide the planning directly, leaving feasibility out of the symbolic representation entirely and learning to avoid infeasible actions through policy rollouts. Their target Q-values are derived from successful plans produced by an oracle planner and scale with the number of remaining steps, so their training data contains no dead-ends and no infeasible actions. REAP mostly learns from dead-end signals during rollouts. Silver et al. [25] (PLOI) rank object importance from demonstrations in order to plan on a problem with a reduced number of objects, which is efficient when few objects matter but degrades when most objects are genuine occluders. Cieślar et al. [26] predict the refinability of a given plan skeleton to guide continuous parameter search. We assume no such skeleton, but instead learn a policy that guides the planning. Furthermore, we are agnostic to the controller, assuming we only need to select a high-level action, and not how to realize it. Chitnis et al. [27] learn symbolic operators from demonstrations for both high-level planning and low-level sampling, whereas we learn action values over pre-defined operators without low-level information.

Object Rearrangement. REAP's evaluations adapt environments from prior rearrangement benchmarks: confined-shelf rearrangement [28], multi-layer shelf retrieval [29], and multi-level stacking [30]. Adjacent work on non-prehensile manipulation among movable obstacles [31] shares our cluttered, geometrically constrained setting. REAP focuses on prehensile actions.

Other Learning Approaches in TAMP. SPOT [32] plans without symbolic discretization, running A* directly over a sequence of transformations of point-cloud scenes. Shah et al. [33] learn relational predicates and PDDL-style operators from unsegmented motion demonstrations, then plan symbolically on instances with longer horizons and far more objects than demonstrated. Further afield, Dhakal et al. [34] learn to weight multiple candidate plans, based on the incurred costs of future tasks in the same environment. Jung et al. [35] distill a motion-planning algorithm into a reactive policy via imitation, eliminating search at deployment. End-to-end visuomotor policies [36] produce motion trajectories directly without an explicit planning step.

3 Method

REAP is a framework for learning general heuristics for rearrangement problems with hidden geometric preconditions. These hidden preconditions are not learned explicitly in syntactic form but

implicitly in a Q-value function that guides the search for feasible plans. The framework consists of three main components: (i) a symbolic simplified abstraction of the rearrangement problem in the form of a classical planning domain, (ii) a relational graph neural network heuristic trained via reinforcement learning and feedback from a feasibility oracle that results in a learned Q-value function, and (iii) a greedy best-first search procedure that uses the learned Q-function as a heuristic to guide the search for a feasible plan. The learned Q-value anticipates the geometric feasibility of actions in the symbolic planning state that must provide a suitable abstraction of the configuration space. As a result, the search for plans that are valid in the explicit planning abstraction while complying with the implicit geometric constraints is informed by the two dimensions of the combined task and motion problem. We describe the components of REAP below. Figure 1 visualizes the three components on an open-tabletop instance.

3.1 Generalized Policies with Hidden Geometrical Preconditions

The *symbolic abstraction* is expressed as a PDDL/STRIPS planning problem $P = \langle D, I \rangle$ where $D = \langle Pred, Act \rangle$ is a planning domain with predicates $Pred$ and action schemas Act, and I is the instance information $I = \langle O, Init, G \rangle$ where O is a set of objects (constants), while $Init$ and G are sets of ground atoms formed from the domain predicates and the objects, the first representing the initial state of the problem P, and G representing the goal condition. Coupled with the PDDL specification is a simulation environment with a feasibility oracle for every symbolic action; in all our experiments the oracle is instantiated by a sampling-based motion planner. The world is modeled as a set of fixed and movable objects. The latter can be moved across one or multiple grids (2D or 3D) with the help of a fixed robot arm. The predicate set $Pred$ models the discrete physical relations: occupancy of cells or support surfaces, gripper state, and support relations between objects. The action schemas Act include actions of the form `pick(obj)` and `place(obj, target)` or suitable variations. The ground actions in an instance are obtained by instantiating the action schemas with the objects. A ground action a is applicable in state s, written $a \in A(s)$, iff $\textsc{pre}(a) \subseteq s$, and yields the successor $s' = f(a, s) = (s \setminus \textsc{del}(a)) \cup \textsc{add}(a)$. A plan for P is a sequence $a_0, \ldots, a_n$ with $a_i \in A(s_i)$, $s_0 = Init$, and $G \subseteq s_{n+1}$. Here $\textsc{pre}(a)$, $\textsc{del}(a)$ and $\textsc{add}(a)$ are the standard STRIPS precondition, delete and add lists of the action schema.

In addition to the symbolic abstraction $P = \langle D, I \rangle$, our framework assumes that there is a hidden function $X(s)$ that represents the geometrically feasible actions in s. We check whether an applicable action $a \in A(s)$ is (geometrically) feasible; namely, $a \in X(s)$, by calling a motion planner. This assumes that the symbolic state s provides a suitable abstraction of the space. The plan $a_0, \ldots, a_n$ above for P is then feasible if each action a_i in the plan is feasible in state s_i where it is executed, namely $a_i \in X(s_i)$. The symbolic abstraction is intentionally a coarse over-approximation of geometric reality: we model only cell occupancy and, in 3D settings, support relations. A symbolically applicable action may therefore have no collision-free trajectory realizing it (e.g., due to joint limits or obstacles blocking the path). The motion-planner check $a \in X(s)$ is the expensive operation, and REAP is designed to minimize the number of such calls before a feasible plan is found.

3.2 Relational GNN Trained with RL

The parametric Q_θ function maps state-action pairs (a, s) into real values $Q_\theta(s, a) = Q(s, a)$. As in reinforcement learning, $Q(s, a)$ is the estimated cost of taking action a in state s and following the policy thereafter; here that cost is the number of remaining actions to reach the goal. Equivalently, Q_θ is the negated action value of standard Q-learning with a reward of -1 per step. Following Ståhlberg et al. [37, 15], the function is represented as a relational-GNN that takes states s as input, extended with atoms representing the goals and the applicable actions a in s, and outputs all $Q_\theta(s, a)$ values in a single pass over the R-GNN [16]. The weights of the GNN are learned via a deep-Q-learning algorithm [38] but with the greedy rollouts that accommodate different forms of exploration replaced by a systematic best-first search: weighted A* with evaluation function $F(s, a) = g(s) + wQ_\theta(s, a)$, where $g(s)$ is the accumulated cost to reach s, and $Q_\theta(s, a)$ is the cost estimate of reaching the goal from s by starting with the action a. Since the learned Q-values are not admissible

lower bounds, we add an exploration component to WA*: with probability ε the search selects a random node from the OPEN list — the frontier of generated but unexpanded nodes maintained by the best-first search — rather than the best-scoring one, and expanded states are placed in a replay buffer for Q-value updates [21]. Encoding states and goals as atoms enables generalization to new instances; systematic exploration enables sparse goals and long plans [39, 40]. Since a single pass through the R-GNN yields the $Q_\theta(s, a)$ for all the applicable actions a in a state, the nodes in the search do not represent states s, but state-action pairs (s, a), which implicitly encode the successor states $s' = f(a, s)$ [20].

During *training*, $w = 2$ in the WA* search, while at *test time*, a greedy best-first search is used instead that discards the accumulated costs $g(s)$ and only uses the $Q_\theta(s, a)$ estimates. The search expands the (s, a) pair in the priority queue with the lowest evaluation by applying the action a in s and obtaining the resulting state s' and the $Q_\theta(s', a')$ values for the actions a' applicable in s'. Before this expansion, we check if s is a goal state, and if not, if the action a is executable in s, i.e., if $a \in X(s)$. Feasibility verdicts are cached, so a violation at one (s, a) is reused on subsequent visits. Early in training, most searches fail and dead-end signals from those failures therefore dominate the learning signal.

At test time, we also evaluate a purely greedy execution variant, REAP-GREEDY, that commits to $\arg\min_a Q_\theta(s, a)$ at each step without backtracking. Problems solved by REAP-GREEDY are thus solved greedily, without any search, illustrating the power of the learned heuristic. In both variants every action is verified by the feasibility oracle in the state where it is expanded, so the learned heuristic determines only *which* actions are checked first, never whether an unverified action can enter a plan. REAP-GBFS, on the other hand, backtracks and can recover from errors in the learned heuristic. It is probabilistically complete on the finite symbolic state space, given a probabilistically complete motion planner; the finite RRT budget and per-problem timeout used in our experiments do not preserve that guarantee.

The same R-GNN architecture is used across all five environments in Section 4.1, each with its own set of predicates, actions, and object types; layer count, embedding size, and activation are detailed in Appendix A.

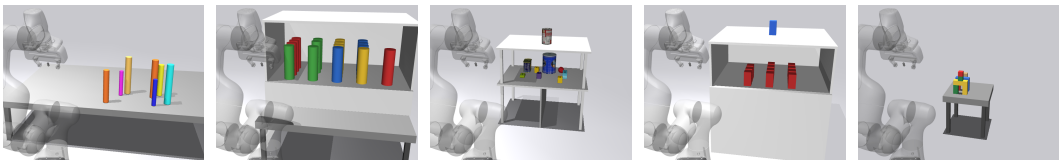
4 Experiments

We evaluate REAP on five tabletop rearrangement environments with different hidden-precondition and task structures, and compare against classical search baselines and an integrated task-and-motion-planning solver. REAP's learned heuristic generalizes out-of-distribution along multiple axes, and that the same architecture and training recipe can be applied across all five environments.

4.1 Setup

Simulator and Robot. All experiments use a single Franka Emika Panda arm in MuJoCo [41], with inverse kinematics solved by MINK [42] and collision-free trajectories planned by a motion planner using linear interpolation and RRT* [43] calls. The same arm and motion-planner backend are used across all tested environments. Only the scene geometry and the PDDL specifications change to accommodate the task specifications.

Environments. Each environment is specified by a PDDL domain, a simulation environment, and a problem generator producing solvable instances. The hidden geometric preconditions are implicit, following the task layout, the environment geometry, and the robot's kinematics. The evaluation is carried out on five environments. The first is an *open-tabletop rearrangement* setting that serves as the primary evaluation domain: a 10×7 grid of cells on a table with cylinders occupying single cells, focused on object rearrangement, depicted in Figure 2a. The other four environments test different rearrangement settings; Section 4.3 describes each of the four in turn.



(a) open-tabletop (b) confined-shelf (c) multi-level shelf (d) Access-19 (e) 3D stacking

Figure 2: REAP instantiates the same relational-GNN value heuristic, trained with motion-planner feedback over hidden geometric preconditions, across five evaluation environments, each with a different hidden geometric precondition. Gripper clearance on the open table, lateral grasp clearance in the confined shelf, reach into a multi-level shelf, front-facing retrieval past blockers (Access-19), and support structure in 3D stacking. At the same time, REAP also learns the cost-to-go of the underlying planning problems, to guide search towards efficient solutions.

Table 1: Results on 300 open-tabletop problems, all methods under an identical 300 s per-problem cap. *MP / plan* is the mean number of distinct (state, action) motion-planner queries per solved problem, deduplicated within a problem, median in brackets. *Exp./steps* is search expansions normalized by plan length, computed per problem and then averaged. *Checks/exp.* is motion-planner queries per expansion, i.e. how often the method's ranking proposes an action the motion planner rejects. Time is averaged over solved problems (timeouts excluded). REAP rows are mean ± std over 6 seeds. All baseline rows are single runs.

Method	Solve (%)	MP / plan [med.]	Checks/exp.	Exp./steps [med.]	Time (s/solved)
REAP-GBFS	**95.9 ± 0.5**	**15.5 ± 5.1** [3.3]	**1.85**	1.27 [1.00]	**3.6**
REAP-GREEDY	89.4 ± 1.5	5.4 ± 0.8 [3.0]	n/a	n/a	1.8
BrFS	56.7	148.8 [12.0]	7.01	7.26 [3.00]	19.5
h_{FF}	76.7	179.0 [1.0]	5.04	3.62 [1.00]	25.4
SAHS [24]	57.3	3002.9 [2962.0]	n/a	1.31 [1.00]	59.3
PLOI + h_{FF} [25]	73.7	247.4 [2.0]	4.09	8.74 [1.00]	42.2
PLOI + BrFS [25]	54.3	132.2 [2.0]	3.56	12.65 [1.00]	23.5
BT-h [26], *given the plan ordering*	81.7	28.5 [3.0]	n/a	4.25 [1.00]	4.5
PDDLStream (test-stream, ff-astar2)	41.7	1.7 [1.0]	n/a[a]	n/a[a]	5.2
PDDLStream (test-stream, ff-eager)	42.7	2.1 [1.0]	n/a[a]	n/a[a]	2.8
PDDLStream (cell-generator, ff-eager)	43.7	9.8 [1.0]	n/a[a]	n/a[a]	11.5
PDDLStream (informed cells + occlusion axiom)	64.3	23.6 [1.0]	n/a[a]	n/a[a]	17.2

[a] PDDLStream exposes no expansion counter.

Baselines. In each domain, the same symbolic action space, motion-planner interface, and simulation backend are used in all methods to ensure a clean comparison. We compare against BrFS, breadth-first search over the symbolic action space, with the MP filter applied at every expansion, and h_{FF}, greedy best-first search ordered by the domain-independent h_{FF}-heuristic [44]. These isolate the contribution of the *learned* value heuristic: BrFS holds learning and heuristic-design constant at zero, h_{FF} holds learning at zero while introducing a classical domain-independent heuristic. PDDLStream [1] is a widely used TAMP planner that interleaves symbolic search with sampling of continuous action parameters. Four configurations of PDDLStream are tested, varying stream definitions and search strategies; the last combines an informed cell generator with a symbolic occlusion axiom in `pick`'s precondition, and is the strongest formulation we found. We further compare against three learning-based methods, referred to below by the names their authors give them: SAHS [24], which learns a relational value function over a geometrically enriched state representation; PLOI [25], which ranks object importance in order to plan on a reduced problem; and BT-h [26], which predicts the refinability of a given plan skeleton. All three are reimplemented in this environment, against the same motion-planner interface and task distribution. We reimplement each against REAP's symbolic action space, motion-planner interface and training problems. PLOI and SAHS receive only the initial state and goal, as REAP does. BT-h is additionally given the object ordering of a reference plan and refines the placements only. PLOI follows its paper's hyperparameters.

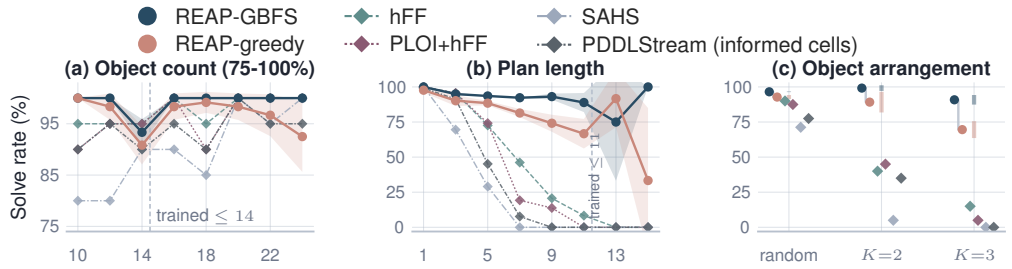


Figure 3: Out-of-distribution generalization along three axes on the open-tabletop environment, for REAP and four baselines. Each policy is trained on problems with up to 14 objects, (optimal) plan lengths up to 11, and random scene placement, and is evaluated on object counts up to 24 (a) and plan lengths up to 15 (b). (c) covers arrangements forcing at least K direct blockers of the OoI to be removed. Vertical dashed lines mark the training-distribution boundary.

4.2 Benchmark Results

All baseline methods and REAP are evaluated on the same 300-problem open-tabletop suite (Table 1) consisting of problems with 3–24 objects and plan lengths from 3 to 15, all verified to be solvable by the motion planner. REAP-GBFS solves substantially more problems than either classical baseline while expanding far fewer states per solved problem. Its median search-overhead ratio is 1.00: on more than half of solved problems REAP-GBFS expands only states that lie on the final plan, reaching the goal without backtracking. The gap widens on problems that require long action sequences, where BrFS and h_{FF} largely time out. Because BrFS and h_{FF} share REAP's symbolic action space and motion-planner filter, the comparison isolates the contribution of the *learned* value heuristic. REAP's margin over h_{FF} isolates what the feasibility signal contributes.

The integrated task-and-motion-planning baseline, PDDLStream, shows a sharper failure mode: it solves $\sim$98% of instances that require no relocation, but its solve rate collapses to $\sim$13% once two to four objects must be relocated and to zero beyond that, reaching a total solve rate below 65%. This pattern holds across all four PDDLStream configurations we evaluate, including the reformulated one.

Across the initial states of the 300 problems, 80% of symbolically applicable actions are geometrically infeasible, the action ranked first by Q_θ is infeasible in only $2.3 \pm 1.5\%$ (six seeds).

4.2.1 Out-of-Distribution Generalization

We evaluate the same value heuristic against three out-of-distribution axes relative to the training set: *object count*, *plan length*, and *object arrangement*. On open-tabletop rearrangement, the policy trained on at most 14 objects with plan lengths up to 11 reaches $95.9 \pm 0.5\%$ solve rate under REAP-GBFS across the full evaluation suite of 300 verified problems within a 300 s per-problem compute cap. The solved subset includes up to 24 objects and plan lengths up to 15 steps. Figure 3 shows that REAP-GBFS stays at or above 89% on every object-count, plan-length, and object-arrangement bucket that contains more than two problems per seed, while the greedy variant degrades on longer plans and harder arrangements.

4.3 Beyond Open-Tabletop Rearrangement

We show that REAP extends with minimal modifications to four further environments, each modified from a published robot manipulation benchmark. Across all four environments REAP keeps the same architecture, training loop, and motion-planner-as-supervisor scheme. Only the PDDL domain and the scene generator are adapted to fit the environment.

Confined-Shelf Rearrangement (Fig. 2b). Wang et al. [28] address grid-based confined-shelf rearrangement where same-colored cylinders must share a column, solved with a classical lazy-

Table 2: REAP-GBFS across the four non-tabletop environments, each reported on its in-distribution and out-of-distribution settings. *exp./steps* is search expansions normalized by plan length; *MP/plan* is defined as in Table 1, with the median in brackets. *Two-phase* runs an extraction policy, then a reload policy. Access-19 uses a 300 s per-problem cap; every problem in the other environments is solved in under 48 s.

Setting	Solve (%)	Mean exp./steps	MP / plan
Confined-shelf rearrangement			
Monotone (in-distribution)	100.0	1.00	7.04 [6]
Non-monotone, 16 objects (OOD)	100.0	1.00	11.24 [12]
Multi-level shelf retrieval			
In-distribution (1–5 blockers)	100.0	1.00	5.52 [5]
OOD (4–10 blockers)	100.0	1.01	7.30 [8]
Access-19			
Single policy, in-distribution ($\leq$ 14 blockers)	86.0 ± 18.4	1.51	68.51 [8]
Two-phase, in-distribution ($\leq$ 14 blockers)[b]	94.0	1.05	17.35 [9]
Two-phase, OOD (18 blockers)[a,b]	100.0	1.62	129.0 [129]
Three-dimensional block stacking			
In-distribution (1–8 blocks)	100.0	1.00	7.14 [3]
OOD (combined structures, 6–13 blocks)	100.0	1.16	22.25 [24]

[a] The eight 18-blocker problems are a single layout up to relabeling of the blockers. [b] Single seed.

evaluation planner under hand-engineered reachability constraints. We re-implement the task as a PDDL domain over a 9×4 grid with 16 cylinders in four color groups, leaving lateral grasp reachability to the motion planner. REAP is trained only on monotone instances (3–9 objects) and evaluated at the same complexity range as theirs: monotone (in-distribution, 3–9 objects) and non-monotone (OOD, 10–16 objects) configurations, the latter requiring an intermediate parking move. Our results (Table 2; §4.5) show that REAP generalizes to the OOD tasks achieving 100% coverage.

Multi-Level Shelf Retrieval and Access-19 (Fig. 2c,d). Ait Bouhsain et al. [29] define the *Access* problem: retrieve an object surrounded by blockers across a multi-layer shelf. They refine it in the *Access-19* instance with 18 blockers arranged in a grid. Their approach predicts feasibility from continuous poses. We re-implement both tasks as distinct PDDL domains, but restricted to front-facing grasps. For multi-level retrieval, REAP is trained on 1–5 blockers (mean 3) and evaluated OOD on 4–10 (mean 7), which is similar to the original but with lower object densities.

A single goal-conditioned policy solves Access-19 in distribution (up to 14 blockers, Table 2) but none of the 18-blocker instances. Splitting the task into two policies, one that extracts the target and one that restores the initial state for the blockers, and executing them in sequence improves solving performance. The composed pipeline solves 25 of 33 held-out layouts with 12–18 blockers, including five 17-blocker layouts and the 18-blocker benchmark instance (a 78-step plan, found in under 45 s), when trained on up to 17-blocker problems. The 18-blocker setting demands subgoal decomposition. Blockers must first be moved away from their goal positions and later restored, which a single policy conditioned only on the final goal is poorly placed to represent.

Three-Dimensional Block Stacking (Fig. 2e). Kulshrestha and Qureshi [30] infer object-dependency graphs to plan multi-level block stacking, evaluated only via pose-matching success. Their problem does not transfer well to a symbolic setting, but the same task structure: build a target structure from a pile of blocks, was the inspiration for our environment. It tests REAP's generalization to 3D grids: a stack table ($10 \times 10 \times 5$). Objects are placed unassembled, goals represent fully assembled multi-part structures. It models multiple symbolic pick and place actions to capture object types and 3D rotations within the grid. REAP is trained on structures with up to 8 blocks and evaluated OOD on combined structures of 6–13 blocks. Table 2 shows that OOD overhead is slightly higher than in the 2D cases, but competitive.

Joint Training and Transfer. One policy trained jointly on open tabletop, confined-shelf and multi-level shelf solves 96.3% of the 300 open-tabletop problems, all 250 confined-shelf, and 94 of 100 multi-level shelf problems, showing that REAP can be trained on multiple environments.

4.4 Ablations

Three RL hyperparameters meaningfully affect open-tabletop training, swept on a 40-problem validator over three seeds per cell. Full per-seed ablation tables and the sample-efficiency study are in Appendix C. The exploration probability ε defined in Section 3.2 is the most impactful: raising it from 0 to 0.2 lifts the mean peak validator solve rate from 87.5 ± 9.4 to 96.7 ± 1.2 and collapses the cross-seed variance. The dead-end cost penalty improves the solve rate monotonically as it grows from -22 to -50. Replay buffer size has no effect on the solve rate. We use the smallest size tried (20) because the trainer loop then sees each collected trace fewer times, biasing training toward recent behavior. ε=0.2, dead-end cost -50, buffer size 20 were used for our experiments.

4.5 Qualitative Analysis

Across environments, the learned policy exhibits interpretable structure beyond what the symbolic domain enforces. On hard *open-tabletop* instances, it puts blockers into the far-south corners and usually starts with the southmost occluder, though plans can run several steps above optimal with occasional non-blocker shuffles. On *confined-shelf*, having been trained only on monotone instances, it generalizes to non-monotone configurations by generalizing parking moves from the monotone rearrangements present during training. On *3D block stacking*, it opens each level with the most-enclosed cell, showing reachability-aware behavior that avoids walling itself off.

5 Limitations

REAP is evaluated entirely in simulation: sim-feasible plans transfer to a real arm only insofar as the simulated kinematics and geometry are faithful, and the symbolic state is assumed provided rather than perceived. Each new environment requires its own simulator, a feasibility oracle supplying per-action verdicts, and a training problem distribution. Per-action motion-planner queries during training rollouts dominate cost, around 24 GPU-hours per run. The action space is restricted to prehensile placements on a discrete grid, leaving non-prehensile manipulation, contact-rich interaction, and articulated-object handling out of scope. The clearest failure we observe is environments that require subgoal decomposition. On the Access-19 environment, the task requires moving blockers away from their goal positions before restoring them, and two policies trained one per phase and executed in sequence do solve it, although the split into phases is designed by hand rather than learned.

6 Conclusion

We presented REAP, a framework that learns a relational Q-value heuristic over a coarse symbolic abstraction with a motion planner serving as a per-action feasibility oracle, and deployed it with greedy best-first search. The same R-GNN configuration and training loop are used across five robot manipulation environments where only the PDDL domain and scene generator change per environment. Geometric feasibility need not be modeled in the symbolic representation to be exploited at search time. A relational value heuristic can carry it implicitly, and that latent geometric knowledge is what produces the scaling and cross-environment behavior we report.

Future directions include replacing the motion planner with a learned low-level controller, a vision-language-action policy, or an RL policy whose success feedback fills the same per-action oracle role. We also want to incorporate more types of manipulation (e.g., non-prehensile), and the explicit ability for the policy to identify and use subgoaling. Closing the sim-to-real gap with physical-robot deployment is an additional research direction.

Acknowledgments

The research has been supported by the Alexander von Humboldt Foundation with funds from the German Federal Ministry for Education and Research. This project was also funded by the German Federal Ministry of Research, Technology and Space (BMFTR) and the Ministry of Culture and Science of the German State of North Rhine-Westphalia (MKW) under the Excellence Strategy of the Federal Government and the Länder. This project has received funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme (Grant agreement No. 885107). Computations were performed with computing resources granted by RWTH Aachen University under project rwth2080.

References

[1] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling. PDDLStream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning. In *International Conference on Automated Planning and Scheduling (ICAPS)*, 2020.

[2] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez. Integrated task and motion planning. *Annual Review of Control, Robotics, and Autonomous Systems*, 4:265–293, 2021.

[3] L. P. Kaelbling and T. Lozano-Pérez. Hierarchical task and motion planning in the now. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2011.

[4] S. Srivastava, E. Fang, L. Riano, R. Chitnis, S. Russell, and P. Abbeel. Combined task and motion planning through an extensible planner-independent interface layer. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2014.

[5] M. Toussaint. Logic-geometric programming: An optimization-based approach to combined task and motion planning. In *International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1930–1936, 2015.

[6] Z. Kingston, M. Moll, and L. E. Kavraki. Sampling-based methods for motion planning with constraints. *Annual Review of Control, Robotics, and Autonomous Systems*, 1:159–185, 2018.

[7] A. M. Wells, N. T. Dantam, A. Shrivastava, and L. E. Kavraki. Learning feasibility for task and motion planning in tabletop environments. In *IEEE Robotics and Automation Letters*, 2019.

[8] D. Driess, O. S. Oguz, J.-S. Ha, and M. Toussaint. Deep visual heuristics: Learning feasibility of mixed-integer programs for manipulation planning. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.

[9] M. Khodeir, B. Agro, and F. Shkurti. Learning to search in task and motion planning with streams. *IEEE Robotics and Automation Letters (RA-L)*, 2023.

[10] Z. Yang, C. R. Garrett, T. Lozano-Pérez, L. P. Kaelbling, and D. Fox. Sequence-based plan feasibility prediction for efficient task and motion planning. In *Robotics: Science and Systems (RSS)*, 2023.

[11] M. Martin and H. Geffner. Learning generalized policies in planning using concept languages. In *Proc. KR*, 2000.

[12] C. Boutilier, R. Reiter, and B. Price. Symbolic dynamic programming for first-order MDPs. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI 2001)*, pages 690–700, 2001.

[13] S. Srivastava, N. Immerman, and S. Zilberstein. Learning generalized plans using abstract counting. In *Proceedings of the 23rd AAAI Conference on Artificial Intelligence (AAAI 2008)*, pages 991–997, 2008.

[14] S. Jiménez, J. Segovia-Aguas, and A. Jonsson. A review of generalized planning. *The Knowledge Engineering Review*, 34, 2019.

[15] S. Ståhlberg, B. Bonet, and H. Geffner. Learning generalized policies without supervision using gnns. In *International Conference on Knowledge Representation and Reasoning (KR)*, 2023.

[16] S. Ståhlberg and H. Geffner. First-order representation languages for goal-conditioned RL. In *AAAI Conference on Artificial Intelligence (AAAI)*, 2026.

[17] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, and W. Zaremba. Hindsight experience replay. In *Proceedings of the 31st Conference on Neural Information Processing Systems (NIPS 2017)*, volume 30, pages 5048–5058, 2017.

[18] S. Nasiriany, V. H. Pong, S. Lin, and S. Levine. Planning with goal-conditioned policies. In *Proceedings of the 33rd Annual Conference on Neural Information Processing Systems (NeurIPS 2019)*, volume 32, pages 14814–14825, 2019.

[19] B. Eysenbach, T. Zhang, S. Levine, and R. Salakhutdinov. Contrastive learning as goal-conditioned reinforcement learning. In *Proceedings of the 36th Annual Conference on Neural Information Processing Systems (NeurIPS 2022)*, volume 35, pages 35603–35620, 2022.

[20] F. Agostinelli, S. S. Shperberg, A. Shmakov, S. McAleer, R. Fox, and P. Baldi. Q* search: Heuristic search with deep q-networks. In *ICAPS Workshop on Bridging the Gap between AI Planning and Reinforcement Learning*, 2024.

[21] M. Aichmüller, Y. Hesse, and H. Geffner. Learning to search and searching to learn for generalization in planning. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2605.25720.

[22] X. Fang, C. R. Garrett, C. Eppner, T. Lozano-Pérez, L. P. Kaelbling, and D. Fox. DiMSam: Diffusion models as samplers for task and motion planning under partial observability. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023.

[23] A. Curtis, N. Kumar, J. Cao, T. Lozano-Pérez, and L. P. Kaelbling. Trust the PRoC3S: Solving long-horizon robotics problems with LLMs and constraint satisfaction. In *Conference on Robot Learning (CoRL)*, volume 270 of *PMLR*, pages 1362–1383, 2024.

[24] B. Kim and L. Shimanuki. Learning value functions with relational state representations for guiding task-and-motion planning. In *Conference on Robot Learning (CoRL)*, 2019.

[25] T. Silver, R. Chitnis, A. Curtis, J. B. Tenenbaum, T. Lozano-Pérez, and L. P. Kaelbling. Planning with learned object importance in large problem instances using graph neural networks. In *AAAI Conference on Artificial Intelligence*, 2021.

[26] B. Cieślar, L. P. Kaelbling, T. Lozano-Pérez, and J. Mendez-Mendez. Learning long-horizon action dependencies in sampling-based bilevel planning. In *Conference on Robot Learning (CoRL)*, 2025.

[27] R. Chitnis, T. Silver, J. B. Tenenbaum, T. Lozano-Pérez, and L. P. Kaelbling. Learning neuro-symbolic relational transition models for bilevel planning. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022.

[28] R. Wang, K. Gao, J. Yu, and K. E. Bekris. Lazy rearrangement planning in confined spaces. In *International Conference on Automated Planning and Scheduling (ICAPS)*, 2022.

[29] S. Ait Bouhsain, B. Merlau, R. Alami, and T. Simeon. Long-horizon task and motion planning with learning-based geometric reasoning, 2025. URL https://hal.science/hal-05392791. HAL preprint.

[30] M. Kulshrestha and A. H. Qureshi. Structural concept learning via graph attention for multi-level rearrangement planning. In *Conference on Robot Learning (CoRL)*, volume 229 of *PMLR*, pages 3180–3193, 2023.

[31] D. M. Saxena and M. Likhachev. Planning for manipulation among movable objects: Deciding which objects go where, in what order, and how. In *International Conference on Automated Planning and Scheduling (ICAPS)*, 2023.

[32] K. Saha, A. Li, A. Rodriguez-Izquierdo, L. Yu, B. Eisner, M. Likhachev, and D. Held. Planning from point clouds over continuous actions for multi-object rearrangement. In *Conference on Robot Learning (CoRL)*, volume 305 of *PMLR*, pages 489–512, 2025.

[33] N. Shah, J. Nagpal, and S. Srivastava. From real world to logic and back: Learning generalizable relational concepts for long horizon robot planning. In *Conference on Robot Learning (CoRL)*, volume 305 of *PMLR*, pages 5362–5434, 2025.

[34] R. Dhakal, D. M. Nguyen, T. Silver, X. Xiao, and G. J. Stein. Anticipatory task and motion planning: Improved rearrangement in persistent continuous-space environments. *IEEE Robotics and Automation Letters (RA-L)*, 2026.

[35] H. Jung, D. Lee, H. Park, J. Kim, and B. Kim. SPIN: Distilling Skill-RRT for long-horizon prehensile and non-prehensile manipulation. In *Conference on Robot Learning (CoRL)*, volume 305 of *PMLR*, pages 1311–1351, 2025.

[36] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023.

[37] S. Ståhlberg, B. Bonet, and H. Geffner. Learning general optimal policies with graph neural networks: Expressive power, transparency, and limits. In *International Conference on Automated Planning and Scheduling (ICAPS)*, 2022.

[38] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. A. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[39] F. Agostinelli, S. McAleer, A. Shmakov, and P. Baldi. Solving the rubik's cube with deep reinforcement learning and search. *Nature Machine Intelligence*, 1(8):356–363, 2019.

[40] L. Orseau and L. H. S. Lelis. Policy-guided heuristic search with guarantees. In *Proceedings of the 35th AAAI Conference on Artificial Intelligence (AAAI 2021)*, pages 12382–12390, 2021.

[41] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033. IEEE, 2012.

[42] K. Zakka. Mink: Python inverse kinematics based on MuJoCo, Feb. 2026. URL https://github.com/kevinzakka/mink.

[43] S. Karaman and E. Frazzoli. Sampling-based algorithms for optimal motion planning. *The International Journal of Robotics Research*, 30(7):846–894, 2011.

[44] J. Hoffmann and B. Nebel. The FF planning system: Fast plan generation through heuristic search. *Journal of Artificial Intelligence Research*, 14:253–302, 2001. doi:10.1613/jair.855.

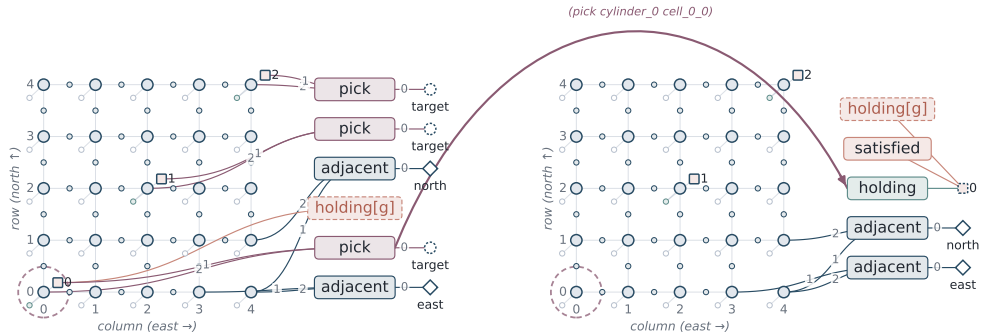


Figure 4: The graph the R-GNN consumes, on a 5×5 open-tabletop instance, before (left) and after (right) the action `pick(cylinder_0, cell_0_0)`. Circles are cell objects laid out on the grid; squares are cylinders; rounded boxes are ground atoms and diamonds are the remaining objects they relate. Edges connect an atom to its arguments and are labeled by argument position, so `adjacent` distinguishes its first cell from its second and the encoding is not permutation-symmetric in the arguments. Applicable actions enter the graph as `pick` atoms over (cylinder, cell) pairs, one per candidate, which is what allows a single pass to score every action in a state. Only occupancy, adjacency and gripper state are represented: no continuous pose, clearance or swept-volume information is available to the network. Comparing the two panels shows what an action changes — the three candidate `pick` atoms are replaced by a single `holding` atom, and the goal atom becomes `satisfied` — and the geometric feasibility of the executed action is nowhere in either graph.

A Training Details

This appendix specifies the graph encoding consumed by the R-GNN (Figure 4) and the two training-loop pieces summarized in Section 3.2: problem sampling and the adaptive per-problem search budget. The feasibility-oracle implementation is described in Appendix B.1.

Problem Sampling. Each worker draws training problems with a weight that depends on the problem's status (*unsolved*, *frontier*, or *solved*), the number of samples t since its last status change, and its consecutive-failure count ϕ. A problem just promoted to *frontier* sits at the boundary of what the current heuristic can solve. It receives a peak weight ω_m for its first τ samples, after which the weight decays geometrically (rate $\delta < 1$) toward a floor $\omega_{\min}$, so attention does not lock onto one difficulty band: $w = \max\big(\omega_{\min},\ \omega_m\, \delta^{\max(0,\, t-\tau)}\big)$. *Unsolved* problems receive a base weight ω_u scaled by their failure count and capped at $\omega_u M$, keeping the learner attempting problems just out of reach: $w = \omega_u \min(1+\alpha\phi,\ M)$. *Solved* problems retain a small residual weight $w = \omega_s$ so they are not forgotten. These values can be used as practical defaults: ω_m=1, τ=20, δ=0.9, $\omega_{\min}$=0.1, ω_u=0.1, α=0.1, M=3, ω_s=0.05.

Adaptive Search Budget. Each problem in the training pool carries an individual search budget B. After five consecutive failures the budget grows by b expansions, up to a cap of κ times the initial value. This focuses learning on the hardest problems the current heuristic can still solve within budget, and relaxes a problem only after repeated failure. We find this adaptive expansion to be crucial for learning to solve long plans. Additionally, a time limit per search can be employed to prevent long searches on hard problems with long motion-planning times.

B Per-Environment Specifications

This appendix specifies the five evaluation environments in a uniform format. Section B.1 factors out what is shared: the simulation backend, robot platform, motion-planning library, and the feasibility-oracle interface used at training and deployment. Each environment-specific subsection then states only what differs from this shared backend.

B.1 Shared Backend

Simulator and Robot. All environments use a single Franka Emika Panda arm in MuJoCo with identical kinematic parameters and collision geometry. Joint limits, end-effector geometry, and gripper finger geometry are fixed across environments.

Motion-Planning Backend. Inverse kinematics is solved with MINK [42] on top of MuJoCo [41]. Collision-free joint-space trajectories are produced by custom RRT and RRT* [43] planners that first attempt linear interpolation (lerp) between IK-solved waypoints and fall back to the sampling-based planner when the lerp fails the collision check. The same planner parameters (iteration budget, step size, goal threshold, smoothing passes) are used across all environments unless explicitly noted.

Feasibility Oracle. The oracle $X(s) \subseteq A(s)$ from Section 3.1 is realized by a single `filter_action(s, a)` entry point that returns FEASIBLE or INFEASIBLE. Verdicts are cached across the search by (s, a) key. INFEASIBLE is returned if any of the following hold: no IK solution exists for the grasp or place pose within joint limits, the RRT planner fails to find a collision-free trajectory within its iteration budget, or the planned trajectory intersects any tracked collision geometry (objects, table surfaces, shelf walls). The entry point is provided by each environment's simulator wrapper.

Environment-Specific Overrides. Each subsection below records only the per-environment overrides relative to this shared backend: the scene geometry, the symbolic abstraction (predicates and action schemas), the problem-instance distribution, and any oracle behavior that differs from the default.

B.2 Open-Tabletop Rearrangement

Scene and World. The environment is a 40×28 cm rectangular table in front of a fixed Franka arm, discretized into a 10×7 grid at 40 mm pitch (Figure 5). Three cylinder classes occupy single cells: fifteen thin cylinders (radius 12.5 mm, height 8 cm), ten medium cylinders (17.5 mm, 10 cm), and five thick cylinders (20 mm, 12 cm). The only symbolic state is which cells are occupied. The hidden geometric structure is the gripper-versus-neighbor interference encountered during top-down approach and retract.

Initial State and Goal Distribution. Training problems sample 3–14 cylinders into random cells, and the goal is a single atom `holding(o)`: retrieve one target cylinder. We use 600 training problems: 480 random placements, 60 instances with a forced-relocation chain of length $K{=}2$, and 60 with $K{=}3$, where the target can be reached only after relocating K blockers in sequence. The 300-problem evaluation suite spans object counts 10–24, plan lengths up to 15, and forced-relocation chains of $K{=}2$ and $K{=}3$. Per-bucket allocations are listed in the Evaluation Protocol below and visualized in Figure 3.

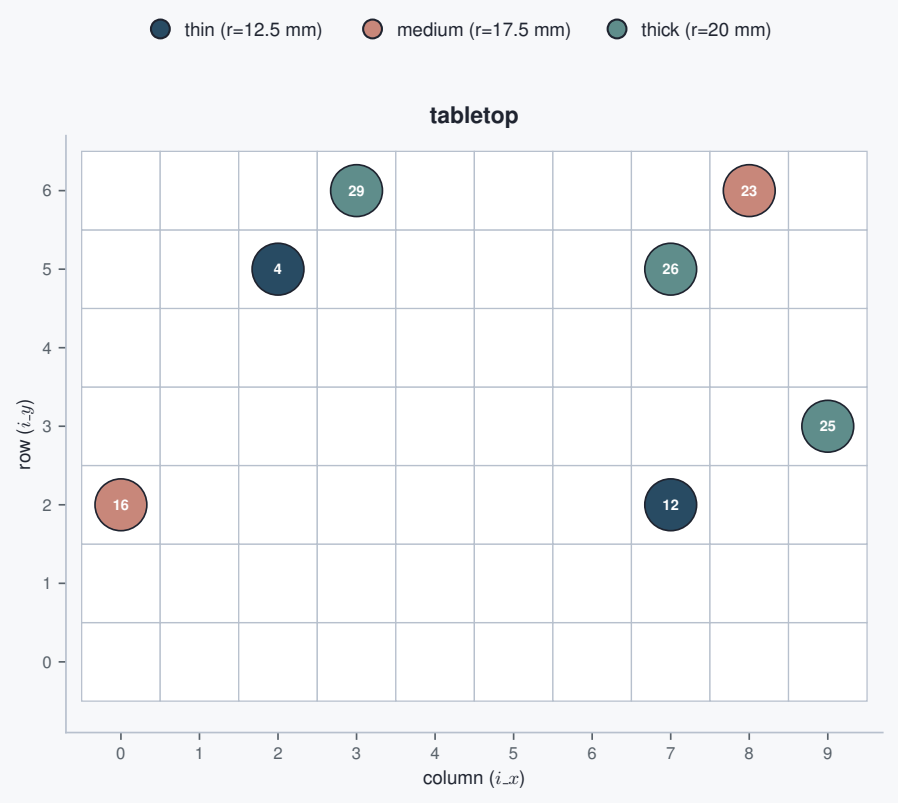
Action and Skill Set. Predicates are per-cell occupancy `occupied(c,cyl)` / `empty(c)`, the grasp pair `holding(cyl)` / `gripper-empty`, and static `adjacent(dir,c,c')` north/east edges. All geometric information available to the R-GNN on this environment is therefore derived from grid occupancy and adjacency alone. Action `pick(cyl,c)`, applicable when the gripper is empty and `c` contains `cyl`, holds the cylinder and frees the cell. Action `put(cyl,c)`, applicable when the gripper holds `cyl` and `c` is empty, releases the cylinder at `c`. The applicable set has size $|A(s)| = N$ in pick states and $|A(s)| = C - N$ in put states, where N is the per-problem cylinder count and $C = 70$ is the grid size.

Feasibility Specification. The symbolic abstraction tracks only per-center-cell occupancy. The oracle realizes $X(s)$ via the shared MINK IK + RRT* pipeline of Section B.1: a side-grasp candidate from the grasp planner, IK at the approach and grasp/place poses, and a collision-free RRT* path

between them, with the carried cylinder modeled as gripper geometry during `put`. A symbolically applicable pick or put is infeasible when no collision-free side-approach corridor to the target cell exists, typically when a south-facing neighbor cylinder occupies the same or an adjacent column.

Dataset and Problem-Instance Statistics. The 600 training problems contain 3 to 14 cylinders each. Optimal plan lengths reach up to 13 actions, with 2.5% of problems at length 11.

Evaluation Protocol. Each problem in the 300-problem suite is solved under a 5-minute compute cap with up to 3 RRT* re-trials per action before declaring infeasibility. These re-trials happen inside the oracle and are distinct from the search-level retries in Table 1. A problem is solved when the search returns a plan whose every action passes the oracle. The motion planner is not seeded: repeating the feasibility evaluation flips about 1.2% of verdicts, so per-problem motion-planner counts are single draws. Table 1 reports REAP rows as the mean ± standard deviation over six seeds; baseline rows are single runs. Figure 3 aggregates over all six seeds per bucket (20 problems per object-count bucket for panel (a), variable bucket sizes for panel (b), and 240/20/40 problems for the random / $K{=}2$ / $K{=}3$ buckets of panel (c)).



Workspace footprint	40 × 28 cm table
Grid regions (resolution over extent, cell pitch)	
table: 10 × 7 over 40 × 28 cm, pitch 40 mm	
Object classes	
thin cyl (×15): $r = 12.5$ mm, $h = 8$ cm, 1 cell	
medium cyl (×10): $r = 17.5$ mm, $h = 10$ cm, 1 cell	
thick cyl (×5): $r = 20$ mm, $h = 12$ cm, 1 cell	

Figure 5: Open-Tabletop. 10 × 7 grid abstraction. The spec table below lists workspace and object dimensions.

B.3 Confined-Shelf Rearrangement

Scene and World. A shelf-shaped cubicle (63×28 cm interior, closed on five sides and open on the $-y$ face) houses up to sixteen identical cylinders (radius 30 mm, height 20 cm, based on bottle dimensions) standing on the shelf floor (Figure 6). The interior is discretized into a 9×4 grid at 70 mm pitch, with the front row aligned to the open face. The cylinders fall into four color groups of four objects each. Color is part of the symbolic state and the goal is defined over it. The hidden constraint is single-side reachability: every grasp must approach from $-y$ through whatever objects currently lie between the gripper and the target.

Initial State and Goal Distribution. The goal in every instance is to sort the cylinders by color, with each of the four columns of the 9×4 grid ending up containing cylinders of a single color. Training instances are *monotone* configurations with 3–9 cylinders, meaning every object can be moved directly to a final column without an intermediate stop. The evaluation suite spans both monotone problems matching the 3–9 training distribution (in-distribution) and *non-monotone* problems at 10, 12, 14, 16 objects (OOD), where at least one cylinder must be parked at an intermediate buffer cell before the color-sorted arrangement is achievable. Headline numbers are in Table 2. The full non-monotone sweep is in Table 3.

Action and Skill Set. Predicates are per-cell occupancy `occupied(c,cyl)` / `empty(c)`, the grasp pair `holding(cyl)` / `gripper-empty`, static grid edges `adjacent(dir,c,c')`, and a static `color-of(cyl,color)` relation. The `adjacent` and `color-of` predicates are not used by the planner. They serve only as structural signals for the R-GNN. Action `pick(cyl,c)` requires the gripper to be empty and `c` to contain `cyl`. Action `put(cyl,c)` requires the gripper to hold `cyl` and `c` to be empty. The grid has $C = 34$ reachable cells, and $|A(s)| = N$ in pick states, $|A(s)| = C - N + 1$ in put states, where N is the per-problem cylinder count.

Feasibility Specification. The symbolic abstraction tracks only per-cell occupancy and grasp state. A symbolically applicable pick or put is infeasible when no collision-free side-approach corridor from the open face to the target cell exists, given the current placement of the other cylinders.

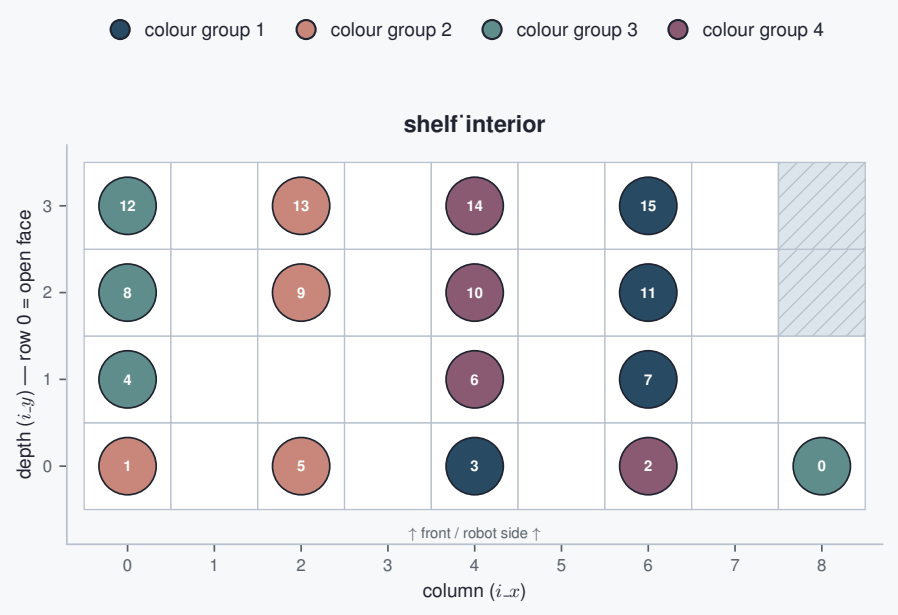
Dataset and Problem-Instance Statistics. The 300 training problems contain 3 to 9 cylinders each. Optimal plan lengths range from 4 to 16 actions.

Evaluation Protocol. Each evaluation set contains 50 problems and is run under a 5-minute compute cap per problem. The in-distribution set holds out 50 monotone problems matching the 3–9 training distribution. The non-monotone OOD sets contain 50 problems each at 10, 12, 14, and 16 cylinders. Table 2 reports the in-distribution row and the 10-cylinder non-monotone OOD row. Table 3 reports the full 10/12/14/16 non-monotone sweep.

Non-Monotone OOD Scaling. Table 2 reports the hardest setting (16 objects). The full non-monotone sweep at 10, 12, 14, 16 objects is reported in Table 3. REAP-GBFS solves 100% at every object count with essentially constant search-overhead ratio and a mild growth in motion-planner queries per plan (10.7 $\rightarrow$ 11.2 from 10 to 16 objects).

Table 3: REAP-GBFS on confined-shelf non-monotone instances, by object count. The model is the single run reported in Table 2. Only the object count varies across rows.

Object count	Solve (%)	Mean exp./steps	MP / plan
10	100.0	1.00	10.72 [11]
12	100.0	1.00	10.96 [12]
14	100.0	1.00	11.04 [12]
16	100.0	1.00	11.24 [12]



Workspace footprint	63×28 cm (closed cubicle, open $-y$ face)
Grid regions (resolution over extent, cell pitch)	
shelf_interior: 9×4 over 63×28 cm, pitch 70 mm	
Object classes	
cylinder ($\times 16$, 4 color groups): $r = 30$ mm, $h = 20$ cm, 1 cell	

Figure 6: Confined-Shelf Rearrangement. 9×4 grid abstraction with the $-y$-facing opening (four color groups visible). The spec table below lists workspace and object dimensions.

B.4 Multi-Level Shelf Retrieval

Scene and World. A three-tier shelf is modeled as four disjoint grid regions: a middle deck and a top deck (each 6×5 over 36×30 cm), and two narrower floor segments `floor_left` and `floor_right` (2×5 each over 12×30 cm), all at 60 mm cell pitch (Figure 7). The shelf is closed on five sides and open on the $-y$ face. Each instance contains a single object of interest (OoI) and a varying number of cuboid blockers ($60 \times 60 \times 90$ mm). The OoI and the blockers each occupy one cell. The hidden constraint is that the target's grasp may be obstructed by adjacent blockers and by the shelf walls, and the only feasible approach is from $-y$.

Initial State and Goal Distribution. Training instances place a single OoI at a random cell across the four regions and surround it with 1–5 blockers (mean 3), drawn uniformly. The goal specifies a target cell for the OoI elsewhere in the shelf. The agent must therefore relocate whatever blockers obstruct either the OoI's grasp approach or its placement at the target. The OOD evaluation set keeps the same goal structure but increases blocker count to 4–10 (mean 7), pushing the shelf toward higher density. Headline numbers are in Table 2.

Action and Skill Set. Predicates are per-cell occupancy `occupied(c,obj)` / `empty(c)`, the grasp pair `holding(obj)` / `gripper-empty`, and static intra-region edges `adjacent(dir,c,c')`. The `adjacent` predicate is not used by the planner. It is a structural signal for the R-GNN. Action `pick(obj,c)` requires the gripper to be empty and `c` to hold `obj`. Action `put(obj,c)` requires the gripper to hold `obj` and `c` to be empty. The applicable set has size $|A(s)| = M$ in pick states and $|A(s)| = C - M + 1$ in put states, where M is the number of active movables and $C = 68$.

Feasibility Specification. The symbolic abstraction tracks per-cell occupancy and grasp state. A symbolically applicable pick or put is infeasible when no collision-free side-approach corridor to the target cell exists, given the placement of the other movables on the same shelf level.

Dataset and Problem-Instance Statistics. The 300 training problems contain 1 to 5 active blockers each (modal 3). The shelf has $C = 68$ cells, fixed across all problems.

Evaluation Protocol. Each evaluation set contains 50 problems and is run under a 5-minute compute cap per problem. The in-distribution set holds out 50 problems matching the 1–5 training distribution. The OOD set contains 50 problems at 4–10 active blockers, extrapolating up to twice the training maximum. Table 2 reports both rows.

B.5 Access-19

Scene and World. Access-19 is a high-density closed cubicle with an open $-y$ face and an open top deck (Figure 8). The cubicle is discretized into two regions at 60 mm pitch with edge exclusions, yielding $C = 96$ reachable cells: `shelf_interior` (47 cells in the enclosed compartment) and `shelf_top` (49 cells on the open deck above). All blockers and the OoI are tall cuboids of bounding-box $40 \times 40 \times 80$ mm, each occupying one cell.

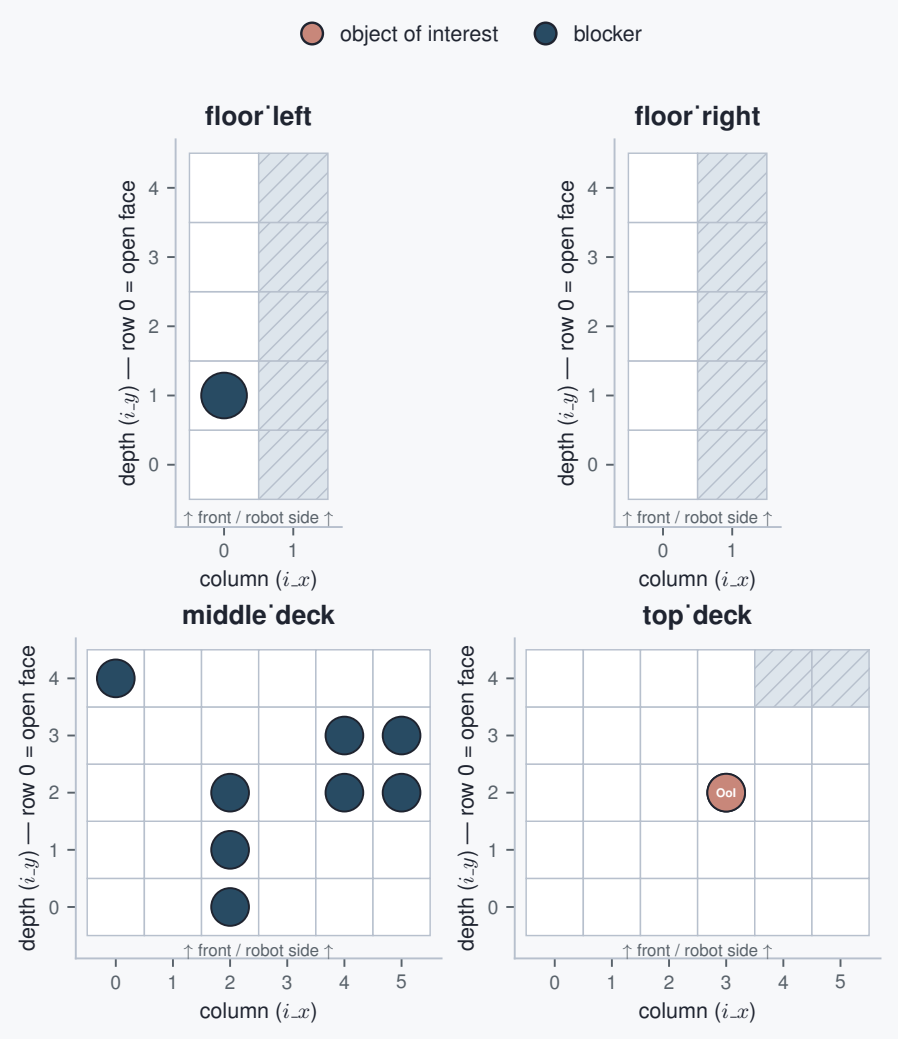
Initial State and Goal Distribution. The OoI is placed deep inside the interior among 0 to 18 blockers, and the goal relocates it to a fixed top-deck cell. Two task types share this scene: in *extraction* problems the goal is the OoI cell alone, while *return* problems additionally require every blocker to be restored to its initial cell, so blockers must first be moved away from their goal positions and later put back. The 18-blocker configuration is adapted from Ait Bouhsain et al. [29] and forces the longest relocation sequences before the OoI becomes graspable. With the OoI goal cell fixed, the shelf admits a single 18-blocker layout up to relabeling of the blockers, so the eight 18-blocker evaluation files are one problem.

Action and Skill Set. Predicates and the `pick` / `put` schemas are identical to the multi-level shelf retrieval environment (Section B.4), with one addition: training uses the derived predicate `blocking(obj,c)`, which holds when `obj` occupies a cell strictly in front of `c` in the same cube column. It is computed symbolically from the other predicates, requires no motion-planner call, and is consumed only by the R-GNN as a structural occlusion signal, not by the planner.

Feasibility Specification. The symbolic abstraction tracks per-cell occupancy and grasp state. A symbolically applicable pick or put is infeasible when no collision-free palm-$+y$ side-approach corridor to the target cell exists, given the placement of the other movables. Occlusion is checked against the neighboring cube columns on both sides, and a put is also infeasible when it would leave two cuboids touching laterally in the same row of the top deck.

Dataset and Problem-Instance Statistics. The single goal-conditioned policy is trained on 358 problems with 0 to 14 blockers (207 of them return problems; reference plans of 2–58 steps, median 6). The two-phase pipeline below trains each policy on its own data.

Two-Phase Pipeline. The pipeline splits a problem into extraction and reload and assigns one policy to each, with the same architecture and training loop as all other environments. The *extraction policy* is trained on 639 extraction problems with 0 to 18 blockers, half of them start-state prefixes of long training plans: replaying the first steps of a reference plan and keeping the reached state as a new problem with 8, 16 or 24 steps remaining. The *reload policy* is trained on 2,288 reload problems with 1 to 17 blockers. About a third start from post-extraction shelves produced by the extraction policy, by earlier extraction policies and by an oracle; the rest are prefixes of reload plans built the same way. Every problem whose target layout occurs in any evaluation set is removed from its training and validation data, so all evaluation layouts are unseen by it and the 18-blocker layout

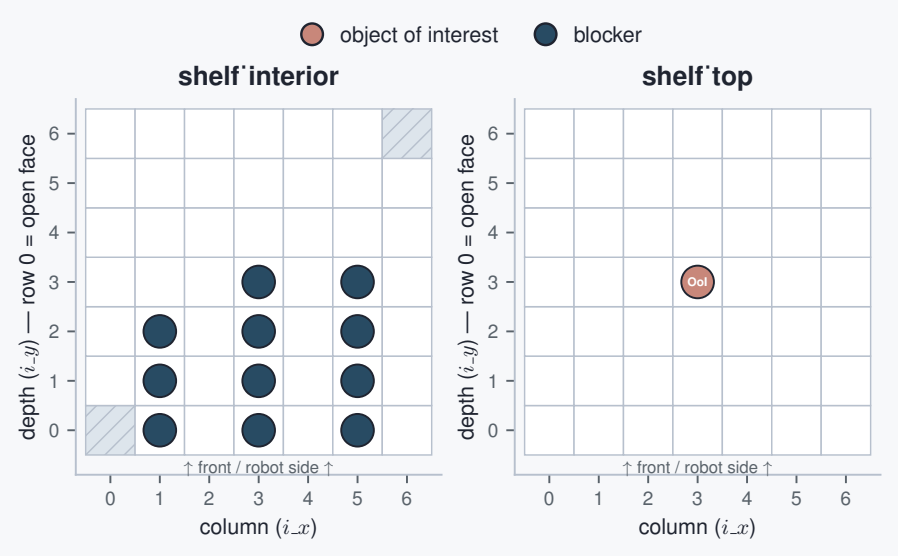


Workspace footprint	3-tier shelf, decks 60×45 cm (open $-y$ face)
Grid regions (resolution over extent, cell pitch)	
floor_left: 2×5 over 12×30 cm, pitch 60 mm	
floor_right: 2×5 over 12×30 cm, pitch 60 mm	
middle_deck: 6×5 over 36×30 cm, pitch 60 mm	
top_deck: 6×5 over 36×30 cm, pitch 60 mm	
Object classes	
box blocker + 1 OoI: half-extents $30 \times 30 \times 45$ mm ($60 \times 60 \times 90$ mm box), 1 cell	

Figure 7: Multi-Level Shelf Retrieval. Grid abstraction over the four disjoint regions (`floor_left`, `floor_right`, `middle_deck`, `top_deck`). The spec table below lists workspace and object dimensions.

is one blocker beyond its training range. Its training search expands one node per step rather than a batch of 64, since batched expansion caps search depth at the budget divided by the batch size, which is shorter than reload plans of 30 or more steps. The reported reload checkpoint is the last best save on its own 71-problem validation set (single seed). At deployment the extraction policy is run with the OoI cell as the only goal, and the reload policy then searches from the reached state towards the full goal, both within one per-problem cap; the concatenated plan is replayed from the original initial state through the feasibility check.

Evaluation Protocol. In-distribution evaluation uses the 100-problem validation set (0–14 blockers; 78 extraction and 22 return problems). The held-out extrapolation set has 30 problems: 22 with up to 14 blockers (15 extraction, 7 return) and the eight 18-blocker return files. Generalization to unseen layouts is measured on 33 return layouts with 12–18 blockers, drawn at the fixed OoI goal and deduplicated by isomorphism. All Access-19 results use the 300 s per-problem cap of the open-tabletop suite (Table 2, Section 4.3).



Workspace footprint	closed cubicle + top deck (open $-y$ face)
Grid regions (resolution over extent, cell pitch)	
shelf_interior: 7×7 over 42×42 cm, pitch 60 mm	
shelf_top: 7×7 over 42×42 cm, pitch 60 mm	
Object classes	
cube: half-extents $20 \times 20 \times 40$ mm ($4 \times 4 \times 8$ cm), 1 cell; 11 red blockers + 1 blue OoI	

Figure 8: Access-19. Grid abstraction over `shelf_interior` and `shelf_top`, showing the OoI (blue) buried among the blockers (red). The spec table below lists workspace and object dimensions.

B.6 Three-Dimensional Block Stacking

Scene and World. Two free-standing tables share the Franka envelope (Figure 9). The *parts table* sits behind the arm (centered at $(0, -0.45, 0)$, 65 cm $\times$ 65 cm) with a 2D grid of 15×15 cells where blocks rest flat at table height. The *stack table* sits in front of the arm (centered at $(0, +0.50, 0)$, 50 cm $\times$ 50 cm) with a 3D grid of $10 \times 10 \times 5$ cells modeled as five stacked 2D regions (`stack_L0` through `stack_L4`) spaced one cube-edge apart in z. Both grids have cell pitch 30 mm, matching the block edge, so blocks pack tightly with no inter-cell gap.

Initial State and Goal Distribution. Each training problem declares a fixed pool of 38 blocks across the cube. Only 1 to 8 of them are referenced in the symbolic predicates of any given training problem. The remaining blocks are hidden in the simulator and do not participate in the task. The goal specifies target positions and orientations for the referenced subset on the stack table. The OOD evaluation set probes generalization to *combined structures* of 6–13 blocks, where two or more simpler target structures are composed into a single more complex assembly. The agent must plan a feasible build order respecting support and gripper-clearance constraints. Headline results are in Table 2.

Action and Skill Set. Three block shapes: 1×1 cubes (30 mm edge), 2×1 oblongs ($60 \times 30 \times 30$ mm), and 3×1 longs ($90 \times 30 \times 30$ mm). Oblongs and longs each admit three valid placement orientations: *flat-x* (cells along $+x$ at the same level), *flat-y* (cells along $+y$ at the same level), and *upright* (cells at the same (i_x, i_y) stacked along z). The upright orientation is valid only on the stack table. Grasps are kinematically constrained: the gripper approaches the long sides of each block face-centered. Cube picks are top-down centered on the cube. Flat-oriented oblong and long picks are top-down with fingers closing perpendicular to the long axis. Upright oblong and long picks are front-facing horizontal with fingers closing on the two longest vertical faces. The PDDL action set covers all three shape classes with pick, put, and in-hand orientation-transform schemas. The transforms mutate a `held-orientation` predicate without changing the world state, modeling free-air gripper rotation between pick and place.

Feasibility Specification. PDDL invariants encode no-overhang structural constraints. Pick preconditions require any directly-above cell to be empty. Put preconditions require the target cell to be empty and either at table level or directly above an occupied cell. Together these guarantee that any block in a stack column is supported by its direct neighbor below or by the table surface.

Gripper-finger clearance against neighbor-cell occupants is *not* encoded in PDDL. It is enforced at runtime by the motion-planner's feasibility check on each candidate (s, a) pair, matching the convention used in the Access-19 environment. The MINK IK and collision query takes the full collision scene including every block at its current placement. The oracle returns 0 when no IK solution exists within joint limits or the planned trajectory collides with any block, table, or workspace boundary. Every grid cell is fully IK-reachable in isolation (100% reachability across both grids in our probe), so the hidden constraint the learned heuristic must anticipate is purely gripper-versus-neighbor interference, not arm reach.

Dataset and Problem-Instance Statistics. The training set contains 670 problems with goals ranging from 1 to 8 target blocks. Reference plan lengths grow from 2 steps for single-block goals to roughly 20 steps for six-to-eight block stacks that require orientation chains.

Evaluation Protocol. The in-distribution evaluation set aggregates 100 problems across five difficulty levels (20 problems per level, 1 to 8 target blocks). The OOD set contains 20 problems with combined structures of 6 to 13 target blocks. Each problem is solved under a 5-minute compute cap. Table 2 reports the in-distribution and OOD rows.

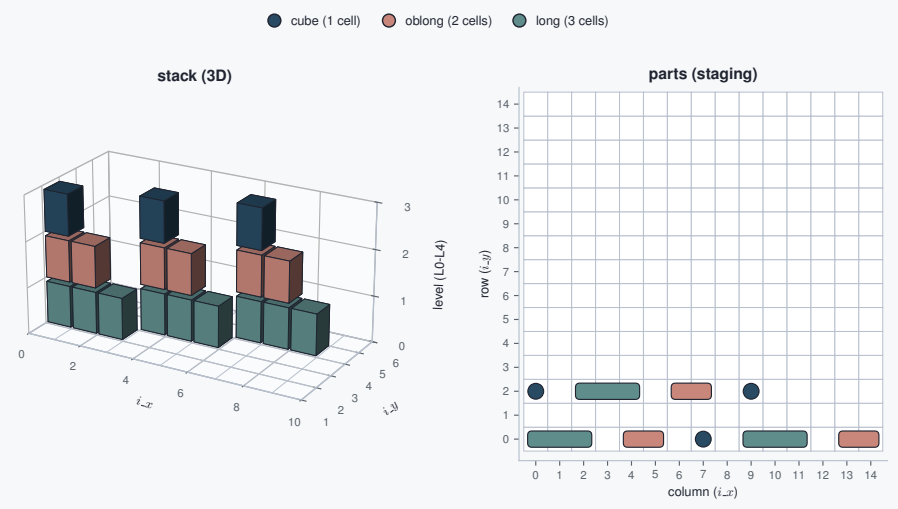
C Open-Tabletop Ablations and Sample Efficiency

All studies in this appendix use the open-tabletop training set with the default domain, three seeds per cell, and a 24-hour walltime cap. For ablations we report the peak validator solve rate (*max*) and the final validator pass before walltime termination (*last*). *max* is the figure a best-checkpoint deployment would select. Most runs were terminated by the walltime cap rather than converging, and the consistent gap between *max* and *last* indicates late-training overfitting, a target for future work (e.g. early stopping or learning-rate decay). The *combined configuration* below, used both for the deployment results in Section 4.2 and for the sample-efficiency study, is ε=0.2, dead-end reward -50, buffer size 20, no worker time limit.

Single-Axis Ablations. Table 4 gives the per-seed single-axis results across exploration rate ε, dead-end reward, worker time limit, search budget multiplier, and replay buffer size.

Combined Configuration. Tables 5 and 6 report the combined-configuration runs, trainer-side and deployment-side respectively, in two worker-time-limit variants.

Sample Efficiency. We study how REAP's performance scales with the size of the training set. The combined configuration is trained on nested subsets of the open-tabletop training set of size



Workspace footprint	two tables; stack $10 \times 10 \times 5$ grid, parts 15×15 grid
Grid regions (resolution over extent, cell pitch)	
stack (per level, $\times 5$): 10×10 over 30×30 cm, pitch 30 mm	
parts: 15×15 over 45×45 cm, pitch 30 mm	
Object classes	
cube: 30 mm edge, 1 cell	
oblong: $60 \times 30 \times 30$ mm, 2 cells (flat or upright)	
long: $90 \times 30 \times 30$ mm, 3 cells (flat or upright)	

Figure 9: Three-Dimensional Block Stacking. Grid abstraction over the two-table workspace: the parts table (15×15) and the five stack-table levels ($10 \times 10 \times 5$). The spec table below lists workspace and object dimensions.

$N \in \{100, 200, 400\}$, three seeds per size, and each run is evaluated with REAP-GBFS on the full 300-problem suite (Table 7). All nine runs trained to convergence. Solve rate is essentially flat across the range, 87.7% at $N{=}100$ and 90.8% at $N{=}400$, with the dip at $N{=}200$ falling within the three-seed standard deviation. Even the smallest training set comes within roughly three points of the largest tested, indicating that REAP does not require a large training set. However, the search-overhead ratio falls monotonically with N ($2.44 \rightarrow 2.17 \rightarrow 1.99$), indicating that a larger set increases search efficiency; motion-planner cost is not monotone in N. The main results in Table 1 instead use the full 600-problem training set and report the mean over six seeds, whereas the per-N rows here are three-seed means.

D PDDLStream Baseline

We compare against PDDLStream [1], an integrated task-and-motion planner that interleaves symbolic search with sampling of continuous action parameters. PDDLStream is run with its adaptive algorithm on the same 300 open-tabletop problems as REAP and under the same 300-second per-problem cap, each scene reconstructed through the tabletop grid transform so that both methods see identical instances. We evaluate three configurations. Two use a fluent test stream for placement feasibility over all target cells, with the Fast Downward planner in its `ff-astar2` and `ff-eager` variants. The third replaces the placement test with a non-fluent *cell generator* that proposes candidate cells lazily, shrinking the planner's optimistic action space. It is the configuration most favorable to PDDLStream on relocation problems. A motion-planner query is counted as one feasibility check (one graspable or placeable stream evaluation), the analog of one expansion under REAP's motion-planner filter.

Table 4: Per-seed single-axis ablation results. Columns are the three seeds' peak validator solve rate, the mean ± standard deviation of the peak (*max*) and final (*last*) validator solve rate, and the trainer iterations reached.

Setting	Seed 1	Seed 2	Seed 3	Mean max	Mean last	Iters
Random exploration ε						
0.0	75.0	97.5	90.0	87.5 ± 9.4	73.3 ± 7.5	1304
0.05	97.5	92.5	92.5	94.2 ± 2.4	88.3 ± 6.2	1159
0.1	95.0	92.5	95.0	94.2 ± 1.2	84.2 ± 3.1	1356
0.2	97.5	97.5	95.0	96.7 ± 1.2	94.2 ± 1.2	1389
Dead-end reward						
-22	80.0	82.5	77.5	80.0 ± 2.0	65.8 ± 3.1	1710
-30	75.0	80.0	92.5	82.5 ± 7.4	67.5 ± 4.1	1719
-50	92.5	87.5	90.0	90.0 ± 2.0	76.7 ± 7.2	1689
Worker time limit (s)						
0	72.5	75.0	75.0	74.2 ± 1.2	64.2 ± 1.2	853
60	77.5	97.5	95.0	90.0 ± 8.9	75.8 ± 8.3	1731
300	75.0	72.5	77.5	75.0 ± 2.0	64.2 ± 1.2	1250
Budget multiplier						
2	97.5	77.5	77.5	84.2 ± 9.4	72.5 ± 5.4	1329
4	75.0	75.0	90.0	80.0 ± 7.1	75.0 ± 8.9	1327
8	90.0	95.0	95.0	93.3 ± 2.4	80.0 ± 6.1	765
Buffer size						
20	77.5	90.0	77.5	81.7 ± 5.9	72.5 ± 3.5	3734
57	72.5	77.5	85.0	78.3 ± 5.1	70.0 ± 2.0	1224
100	75.0	77.5	85.0	79.2 ± 4.2	75.0 ± 4.1	805

Table 5: Combined-configuration runs in two worker-time-limit variants, three seeds each. All runs were terminated by the walltime cap.

Run	Peak (max)	Final (last)	Iters
Worker time limit $= 0$			
Seed 1	95.0	87.5	1372
Seed 2	95.0	90.0	2005
Seed 3	95.0	82.5	1920
Mean	95.0 ± 0.0	86.7 ± 3.1	1766
Worker time limit $= 60\,s$			
Seed 1	90.0	87.5	2526
Seed 2	97.5	77.5	2530
Seed 3	92.5	82.5	1785
Mean	93.3 ± 3.1	82.5 ± 4.1	2280
Overall (6 seeds)	94.2 ± 2.4	84.6 ± 4.4	2023

All three configurations solve 42–44% of the suite (Table 1), but the aggregate rate hides a sharp structural failure. Table 8 groups the 300 problems by reference-plan length, the number of pick-and-place actions a solution requires. Every configuration solves 98–99% of instances that need no relocation, but the solve rate collapses once relocation is required: to 11–23% at two-to-four relocations and to zero beyond. The cell generator helps the relocation band modestly (roughly doubling the solve rate at reference length two to four) but does not change the picture, remaining at or near zero on every forced-relocation dataset (Table 9). PDDLStream is a competent classical TAMP solver. Black-box feasibility gives its symbolic search no signal about *which* object blocks the target, so it fails specifically in the regime where hidden geometric preconditions force long relocation sequences, the regime REAP is designed for.

PDDLStream's per-solved efficiency metrics in Table 1 are averaged over its solved subset, almost entirely no-relocation instances, and so are not directly comparable to REAP's. Table 10 instead

Table 6: Deployment evaluation of the combined-configuration runs on the 300-problem open-tabletop suite, per seed, after the full 24-hour training. In contrast to Table 5, which reports the trainer-side validator solve rate, these are deployment solve rates under REAP-GBFS and REAP-GREEDY. *exp./steps* is the mean search-overhead ratio. *MP/plan* is defined as in Table 1, with the median in brackets. *Plan len.* is mean solved-plan length. One `tl`=0 seed is excluded as overtrained (early stopping did not fire in time), leaving two seeds in that group and three at `tl`=60; the reported spread is over those seeds.

	REAP-GBFS			REAP-GREEDY	
Run	Solve (%)	exp./steps	MP/plan	Solve (%)	Plan len.
Worker time limit = 0					
Seed 2	96.0	1.19	13.1 [3]	90.7	4.6
Seed 3	96.7	1.18	11.5 [3]	89.3	4.2
Mean	96.3	1.19	12.3	90.0	4.4
Worker time limit = 60 *s*					
Seed 1	95.0	1.36	19.8 [3]	86.7	6.3
Seed 2	96.0	1.27	22.8 [3]	91.7	4.6
Seed 3	96.0	1.45	18.2 [4]	89.3	5.0
Mean	95.7 ± 0.5	1.36	20.2	89.2 ± 2.0	5.3

Table 7: Sample efficiency across training-set size N (combined configuration, three seeds per N, evaluated on the 300-problem suite). Solve rate is the mean $\pm$ standard deviation over three seeds. The remaining columns are three-seed means. *exp./steps* is the search-overhead ratio. *MP/plan* is defined as in Table 1, with the median in brackets. Time is per solved problem.

N	Solve (%)	MP / plan	Mean exp./steps	Median exp./steps	Time (s)
100	87.7 ± 2.4	49.8 [5.2]	2.44	1.18	15.2
200	84.6 ± 2.5	36.4 [3.7]	2.17	1.00	11.5
400	90.8 ± 4.7	36.8 [5.0]	1.99	1.00	11.1

restricts the comparison to the problems solved by *both* methods. PDDLStream's solved set is close to a strict subset of REAP's: 125 of the 126 problems solved by the `test-stream/ff-astar2` configuration are also solved by REAP-GBFS. On that shared and predominantly easy set, REAP is at least as motion-planner-efficient as PDDLStream (1.3 versus 2.4 queries per plan), and the cell-generator configuration is roughly six times less efficient. PDDLStream is therefore no more efficient than REAP even on the easy problems it does solve. The REAP-GBFS join uses the seed-2 run of the combined configuration (96% solve rate), as the seed-3 per-problem records were unavailable for the join. This is immaterial to the comparison.

Table 8: PDDLStream solve rate (%) by reference-plan length on the 300-problem open-tabletop suite, for all three configurations. Reference-plan length is the number of pick-and-place actions a solution requires. Longer plans imply more forced relocations.

Reference-plan length	# problems	test/ff-astar2	test/ff-eager	cell-gen.
≤ 1 (no relocation)	122	98	98	99
2–4	46	13	11	23
5–8	88	0	0	1
9+	44	0	0	0

E Execution Under Perturbed Observations

REAP assumes the symbolic state is supplied rather than perceived. To probe how far that assumption can be stretched, we evaluate a trained policy under three perturbations applied at execution time, on the open-tabletop suite. The policy is unchanged; only execution is perturbed.

Table 9: PDDLStream solve rate (%) per evaluation dataset, for the `test-stream/ff-astar2` and `cell-generator` configurations. The `test-stream/ff-eager` configuration matches `ff-astar2` to within one problem per cell.

Dataset	test/ff-astar2	cell-gen.
Object-count scaling		
obj10	45	50
obj12	75	75
obj14	60	60
obj16	80	89
obj18	70	75
obj20	85	95
obj22	100	95
obj24	95	100
Forced relocation		
obj10_plan3	5	10
obj12_plan3	15	15
obj14_plan5	0	0
obj16_plan5	0	6
obj10_plan5_biased2	0	0
obj12_plan7_biased3	0	0
obj14_plan9_biased3	0	0

Table 10: Motion-planner queries per plan on the common solved set, the problems solved by both PDDLStream and the corresponding REAP planner. |both| is the size of that set; *ref.* $|\pi|$ is the mean reference plan length of the PDDLStream configuration's own solved set, i.e. how hard the shared problems are. *MP/plan* is defined as in Table 1. REAP-GREEDY issues fewer queries than every configuration. REAP-GBFS does not: on the two test-stream arms it issues more. Those two arms solve 41.7% and 42.7% of the suite and their solved sets average a reference plan length of 1.1, so the shared set is almost entirely one-action problems, on which a search that also handles the 35 length-9+ problems no configuration solves is necessarily less economical. Where a configuration attempts harder problems the ordering reverses: on C5, the strongest arm at 64.3%, REAP-GBFS is $4.1\times$ cheaper over 190 shared problems.

		vs. REAP-GBFS			vs. REAP-GREEDY		
PDDLStream configuration	ref. $\|\pi\|$	|both|	PDDLStream	REAP	|both|	PDDLStream	REAP
test-stream, ff-astar2	1.1	124	1.6	3.2	114	1.7	1.3
test-stream, ff-eager	1.1	128	2.1	3.2	118	2.2	1.3
cell-generator, ff-eager	1.2	131	9.8	3.2	119	10.0	1.4
informed cells + occlusion axiom (C5)	2.1	190	23.1	5.6	173	24.1	2.6

- **Placement drift.** Each executed `put` lands with Gaussian positional error of standard deviation $\sigma \in \{2, 5, 10\}$ mm.
- **Occupancy misreport.** Each cell's occupancy is flipped with probability $p \in \{0.01, 0.02, 0.05\}$ when the state is read.
- **Execution failure.** Each grasp fails with probability $p \in \{0.02, 0.05, 0.10\}$.

On detecting a discrepancy the system replans once from the observed state. We report the fraction of episodes completing without replanning and, of those that replan, the fraction recovered (Table 11).

Replanning recovers transient errors but not standing ones. A single replan recovers 91–100% of occupancy misreports and 85–100% of execution failures, but only 7–19% of placement-drift failures. The asymmetry follows from what each perturbation does to the world. A misreported cell or a failed grasp is *transient*: the next observation or the next attempt sees the true state, and a replan from that state is as good as the original plan. Drift is a *standing* property of the resulting scene —

Table 11: Execution under perturbation, REAP-GBFS / REAP-GREEDY. *Completed* is the fraction of episodes finishing without a replan; *recovered* is the fraction of replanned episodes that then succeed.

Perturbation	Level	Completed (%)	Recovered (%)
Placement drift			
σ	2 mm	85.4 / 81.6	54.8 / 60.4
	5 mm	71.5 / 73.4	14.6 / 18.8
	10 mm	52.4 / 50.8	6.6 / 7.8
Occupancy misreport			
p	0.01	86.8 / 83.0	100.0 / 91.1
	0.02	85.7 / 87.2	100.0 / 97.1
	0.05	80.8 / 79.2	100.0 / 94.5
Execution failure			
p	0.02	89.5 / 84.0	100.0 / 95.2
	0.05	86.4 / 84.0	97.4 / 95.2
	0.10	75.3 / 77.9	91.5 / 84.5

the object is genuinely a few millimeters off center — and a replan over the idealized grid cannot perceive it, so it walks into the same class of failure again.

This locates the limit precisely. It is a property of the symbolic abstraction, not of the search strategy: REAP-GBFS and REAP-GREEDY degrade almost identically under all three perturbations, despite differing by six points of solve rate on the unperturbed suite. Cell-centered occupancy is what buys the cheap state representation of Section 3, and it is also what makes sub-cell geometric error invisible. Closing this would require the abstraction to carry continuous placement information, or the feasibility oracle to be queried against the observed rather than the assumed scene.